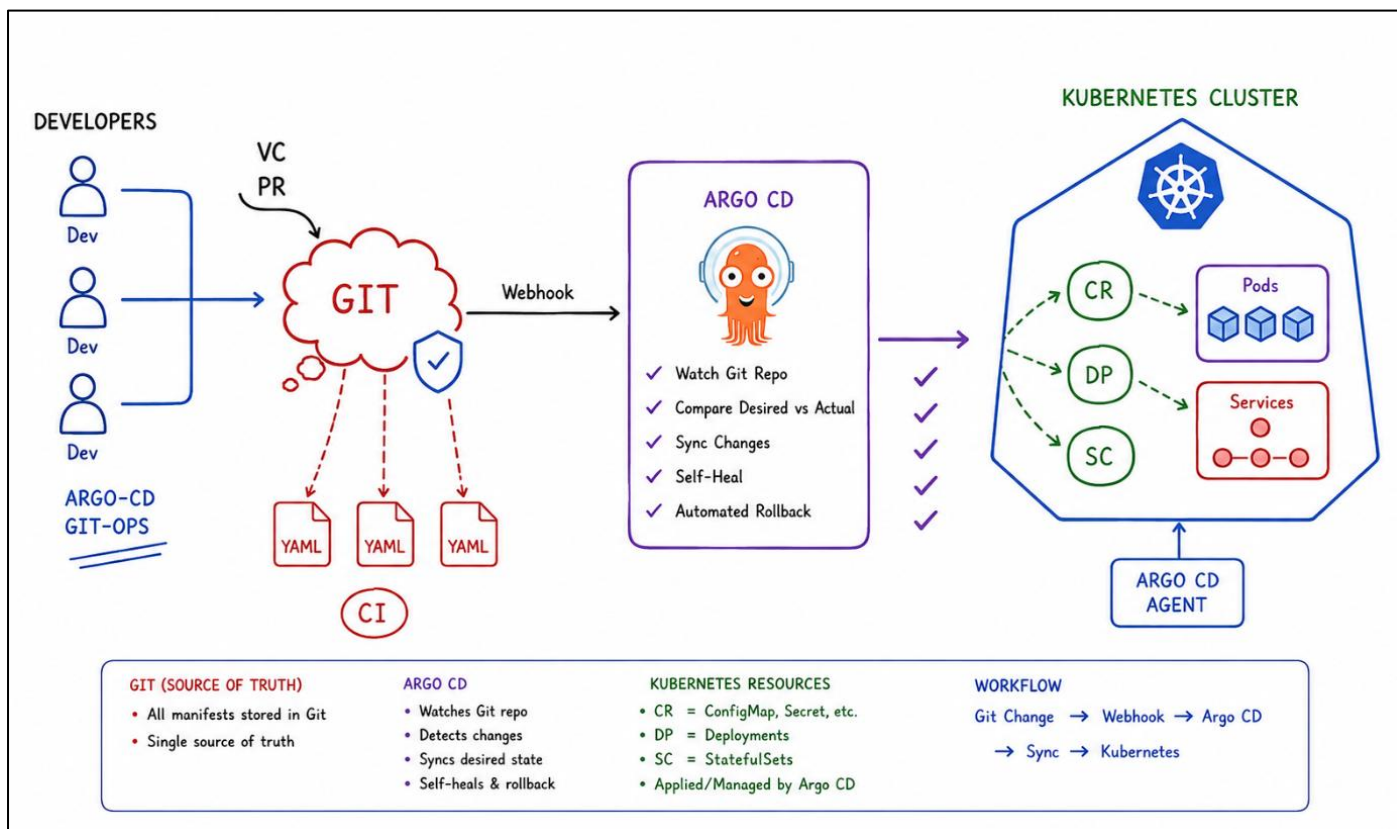




## What is Argo CD?



Argo CD is a **GitOps Continuous Delivery (CD)** tool for Kubernetes. It automatically deploys applications to Kubernetes by reading Kubernetes manifest files stored in a Git repository.



### Simple Definition

Argo CD continuously monitors a Git repository and ensures the Kubernetes cluster matches the **desired state** defined in Git.

## Why Do We Need Argo CD?

### Traditional Deployment

Without Argo CD:



Developer



Git Repository



Jenkins Pipeline



kubectl apply -f deployment.yaml



Kubernetes Cluster

### Problems:

- Manual deployment steps
- Configuration drift
- Hard to track who changed what
- Cluster may differ from Git

### GitOps with Argo CD



Developer



Git Push



Git Repository



Argo CD



Kubernetes Cluster

### Argo CD automatically:

- ✓ Detects Git changes
- ✓ Syncs changes to Kubernetes
- ✓ Maintains desired state
- ✓ Performs self-healing

### Key Benefits of Argo CD



Automated Deployments



Self-Healing (Fixes Drift)



Easy Rollbacks



Audit & Version Control (Git History)



Visibility (UI & Dashboards)



Secure & Reliable



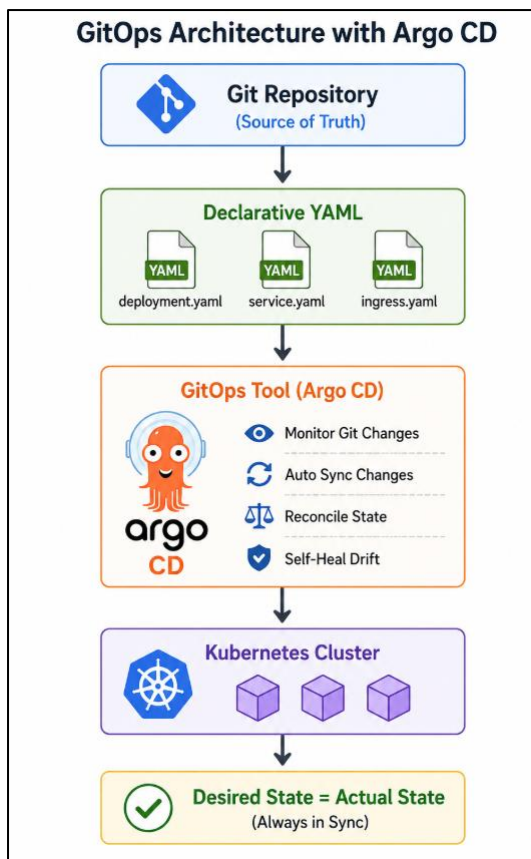
## Interview Answer

Argo CD is a GitOps-based Continuous Delivery tool for Kubernetes. It continuously monitors a Git repository and ensures that the actual state of the Kubernetes cluster matches the desired state defined in Git. It provides automated deployments, self-healing, rollback capabilities, and centralized Kubernetes application management.

## Interview Answer

GitOps is based on four principles:

1. **Git as the Single Source of Truth** – All desired configurations are stored in Git.
2. **Declarative Configuration** – Infrastructure and applications are defined as code.
3. **Automated Synchronization** – Git changes are automatically applied to the cluster.
4. **Continuous Reconciliation** – The system continuously ensures the actual state matches the desired state and self-heals any drift.



## Argo CD with an Amazon EKS

Setting up Argo CD with an Amazon EKS (Elastic Kubernetes Service) cluster involves several steps.

### Prerequisites

1. **Amazon EKS Cluster** : Ensure you have an EKS cluster up and running.
2. **kubectl** : Make sure kubectl is installed and configured to access your EKS cluster.
3. **AWS CLI** : Ensure you have the AWS CLI installed and configured with the necessary permissions.

### Steps:

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ eksctl create cluster --name clusterproject --nodegroup-name b17dng --node-type m7i-flex.large --nodes 4 --managed
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes (main)
```

```
$ eksctl get cluster
```

| NAME | REGION | EKSCTL CREATED |
|------|--------|----------------|
|------|--------|----------------|

|                |           |      |
|----------------|-----------|------|
| clusterproject | us-east-1 | True |
|----------------|-----------|------|

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

\$ kubectl get nodes

| NAME                           | STATUS | ROLES  | AGE   | VERSION             |
|--------------------------------|--------|--------|-------|---------------------|
| ip-192-168-24-83.ec2.internal  | Ready  | <none> | 4m25s | v1.34.8-eks-3385e9b |
| ip-192-168-27-125.ec2.internal | Ready  | <none> | 4m26s | v1.34.8-eks-3385e9b |
| ip-192-168-53-90.ec2.internal  | Ready  | <none> | 4m24s | v1.34.8-eks-3385e9b |
| ip-192-168-63-80.ec2.internal  | Ready  | <none> | 4m25s | v1.34.8-eks-3385e9b |

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

\$ kubectl create namespace argocd  
namespace/argocd created

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

\$ kubectl apply -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml -n argocd

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

\$ kubectl get svc -n argocd

| NAME                                    | TYPE      | CLUSTER-IP     | EXTERNAL-IP | PORT(S)                    | AGE   |
|-----------------------------------------|-----------|----------------|-------------|----------------------------|-------|
| argocd-applicationset-controller        | ClusterIP | 10.100.186.197 | <none>      | 7000/TCP,8080/TCP          | 2m15s |
| argocd-dex-server                       | ClusterIP | 10.100.242.207 | <none>      | 5556/TCP,5557/TCP,5558/TCP | 2m15s |
| argocd-metrics                          | ClusterIP | 10.100.157.33  | <none>      | 8082/TCP                   | 2m15s |
| argocd-notifications-controller-metrics | ClusterIP | 10.100.199.19  | <none>      | 9001/TCP                   | 2m15s |
| argocd-redis                            | ClusterIP | 10.100.246.173 | <none>      | 6379/TCP                   | 2m15s |
| argocd-repo-server                      | ClusterIP | 10.100.242.52  | <none>      | 8081/TCP,8084/TCP          | 2m15s |
| argocd-server                           | ClusterIP | 10.100.118.37  | <none>      | 80/TCP,443/TCP             | 2m15s |
| argocd-server-metrics                   | ClusterIP | 10.100.237.17  | <none>      | 8083/TCP                   | 2m15s |

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

\$ kubectl get svc -n argocd

| NAME                             | TYPE      | CLUSTER-IP     | EXTERNAL-IP |
|----------------------------------|-----------|----------------|-------------|
| argocd-applicationset-controller | ClusterIP | 10.100.186.197 | <none>      |
| 7000/TCP,8080/TCP                |           |                |             |
| argocd-dex-server                | ClusterIP | 10.100.242.207 | <none>      |
| 5556/TCP,5557/TCP,5558/TCP       |           |                |             |
| argocd-metrics                   | ClusterIP | 10.100.157.33  | <none>      |
| 8082/TCP                         |           |                |             |

argocd-notifications-controller-metrics ClusterIP 10.100.199.19 <none>  
9001/TCP 3m4s

argocd-redis ClusterIP 10.100.246.173 <none>  
6379/TCP 3m4s

argocd-repo-server ClusterIP 10.100.242.52 <none>  
8081/TCP,8084/TCP 3m4s

argocd-server LoadBalancer 10.100.118.37  
**a98d5f6c333984dc1aba82d175724314-1758031739.us-east-1.elb.amazonaws.com**  
80:31841/TCP,443:30100/TCP 3m4s

argocd-server-metrics ClusterIP 10.100.237.17 <none>  
8083/TCP 3m4s

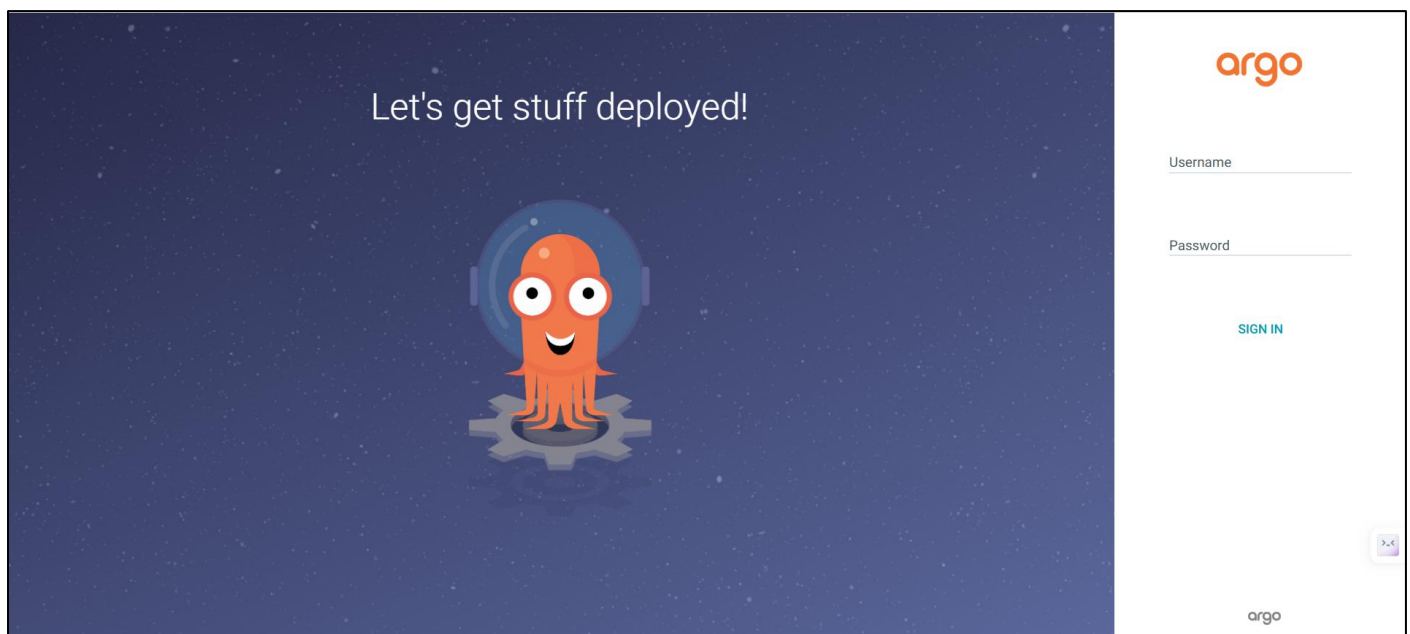
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-  
Devops\_MC\_B17D/Kubernetes (main)

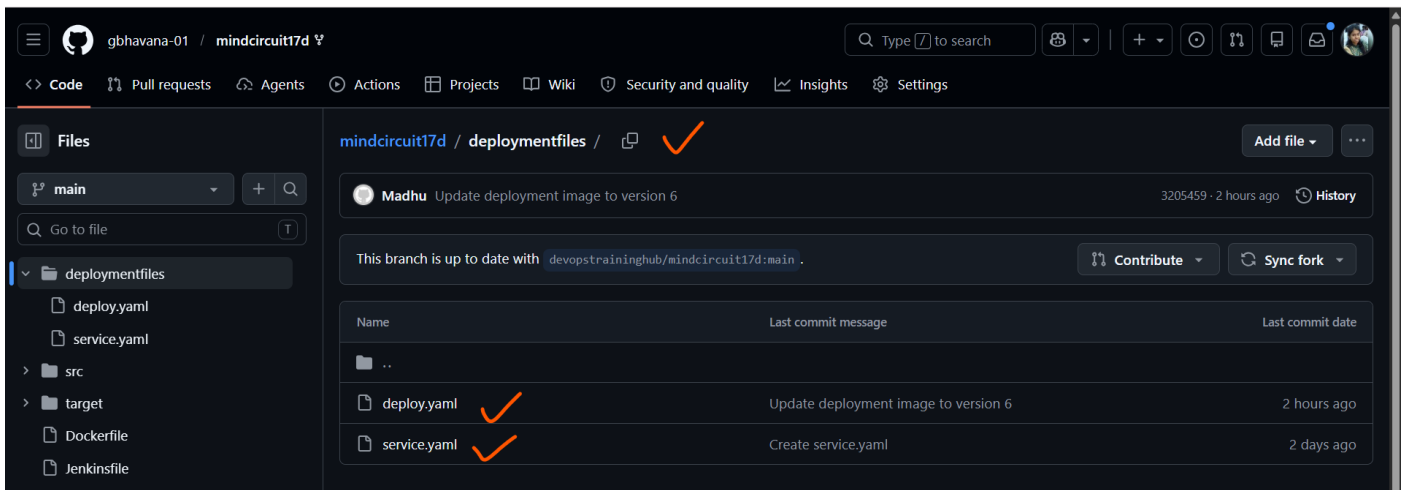
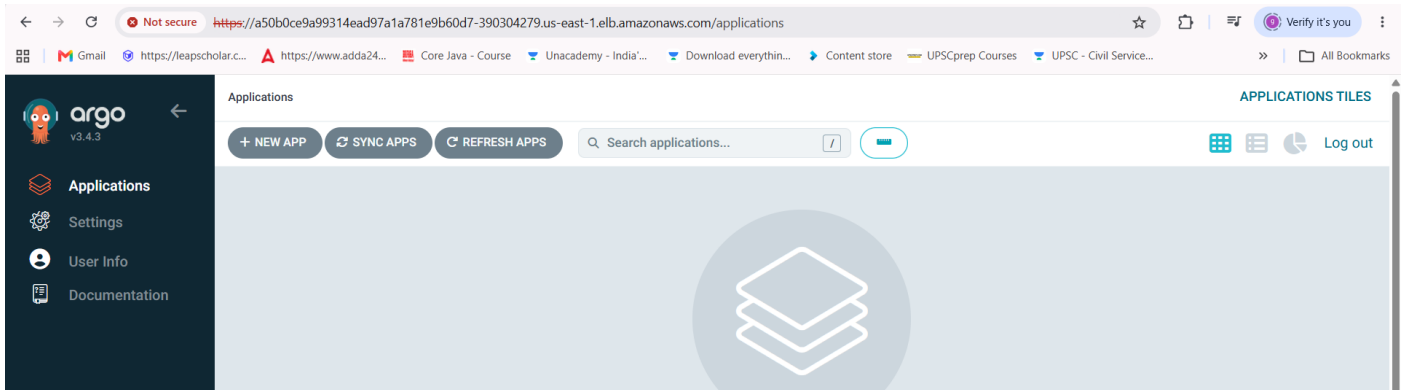
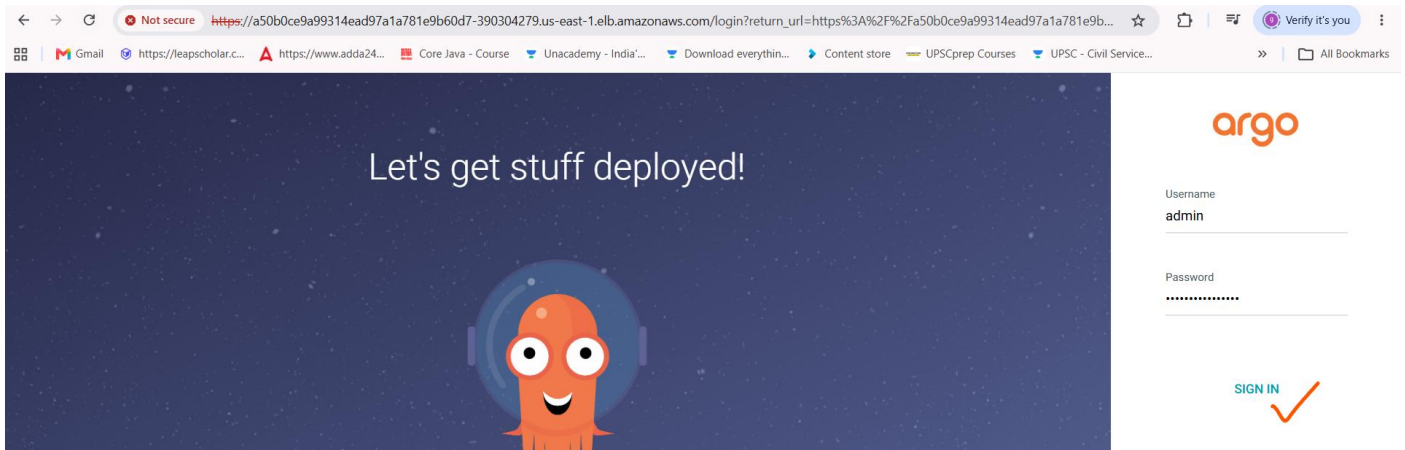
```
$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |  
base64 -d; echo
```

gdQZwo6wMrK5eI5e

**Argo Cd loadbalancer endpoint**

**a98d5f6c333984dc1aba82d175724314-1758031739.us-east-1.elb.amazonaws.com**





argo  
v3.4.3

Applications

Settings

User Info

Documentation

Applications

CREATECANCEL

+ NEW APP

EDIT AS YAML

GENERAL

Application Name  
lab

Project Name  
default

SYNC POLICY

Automatic

☒ ENABLE AUTO-SYNC

☐ PRUNE RESOURCES

☒ SELF HEAL

☐ SET DELETION FINALIZER

SYNC OPTIONS

☐ SKIP SCHEMA VALIDATION

☐ APPLY OUT OF SYNC ONLY

☐ AUTO-CREATE NAMESPACE

☐ RESPECT IGNORE DIFFERENCES

argo  
v3.4.3

Applications

Settings

User Info

Documentation

Applications

CREATECANCEL

+ NEW APP

SOURCE

Repository URL  
https://github.com/gbhavana-01/mindcircuit17d.git

Revision  
HEAD

Path  
deploymentfiles

GIT

Branches

argo  
v3.4.3

Applications

Settings

User Info

Documentation

Applications

CREATECANCEL

+ NEW APP

SOURCE

Repository URL  
https://github.com/gbhavana-01/mindcircuit17d.git

Revision  
HEAD

Path  
deploymentfiles

GIT

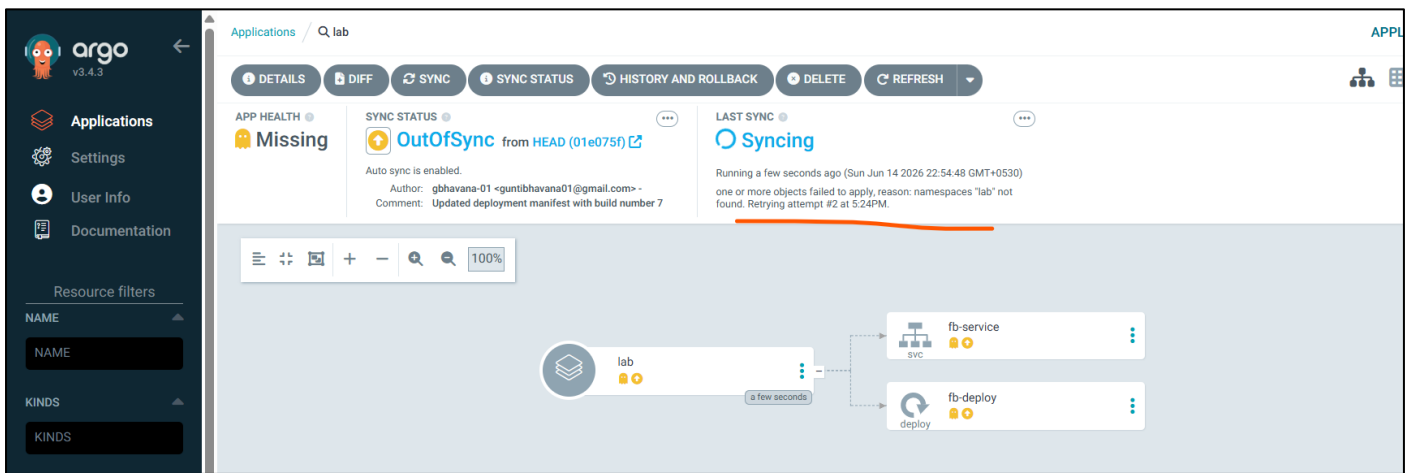
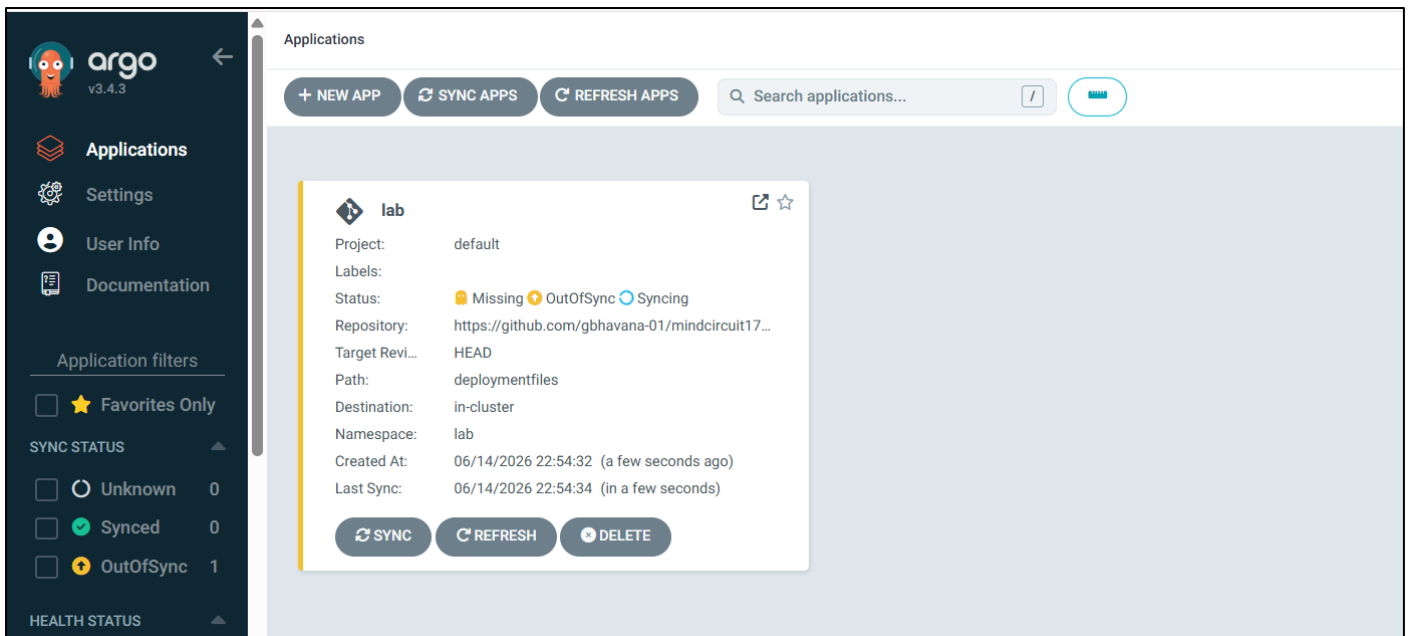
Branches

DESTINATION

Cluster URL  
https://kubernetes.default.svc

Namespace  
lab

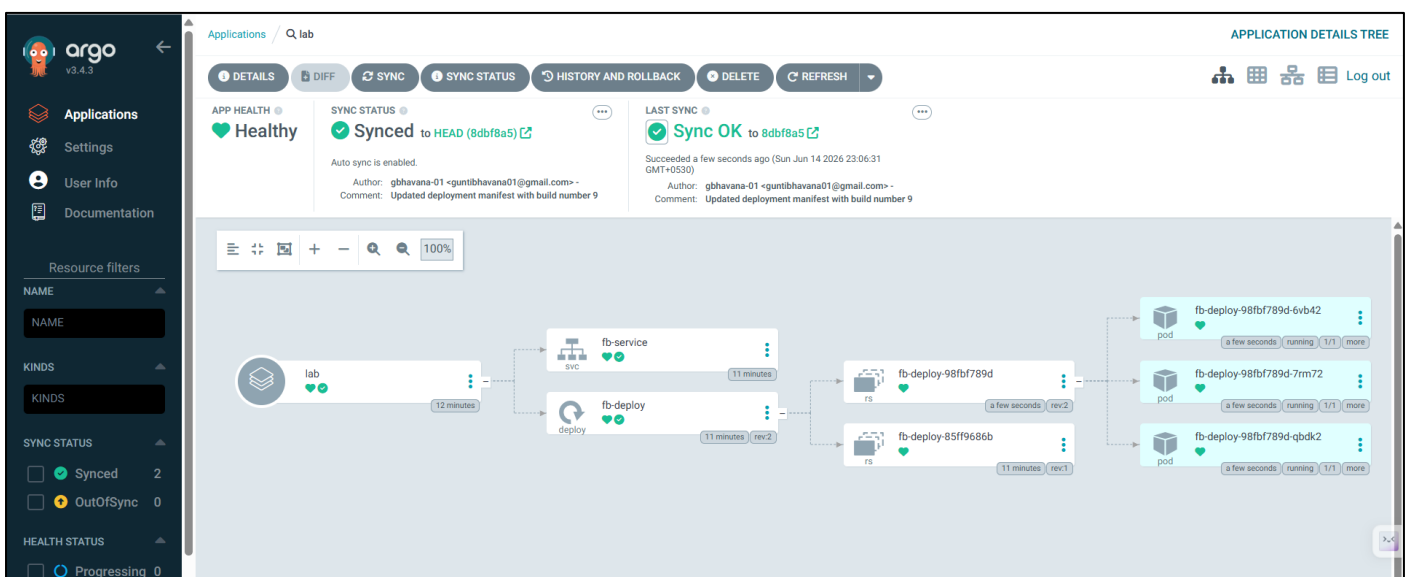
URL

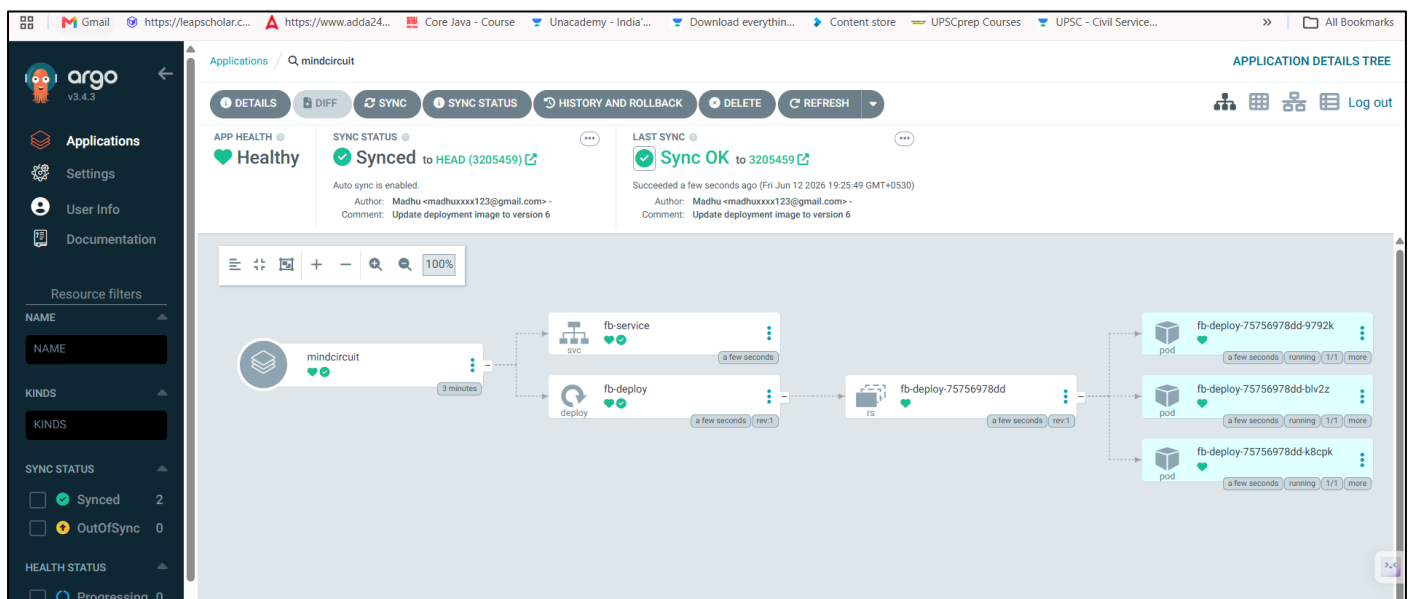
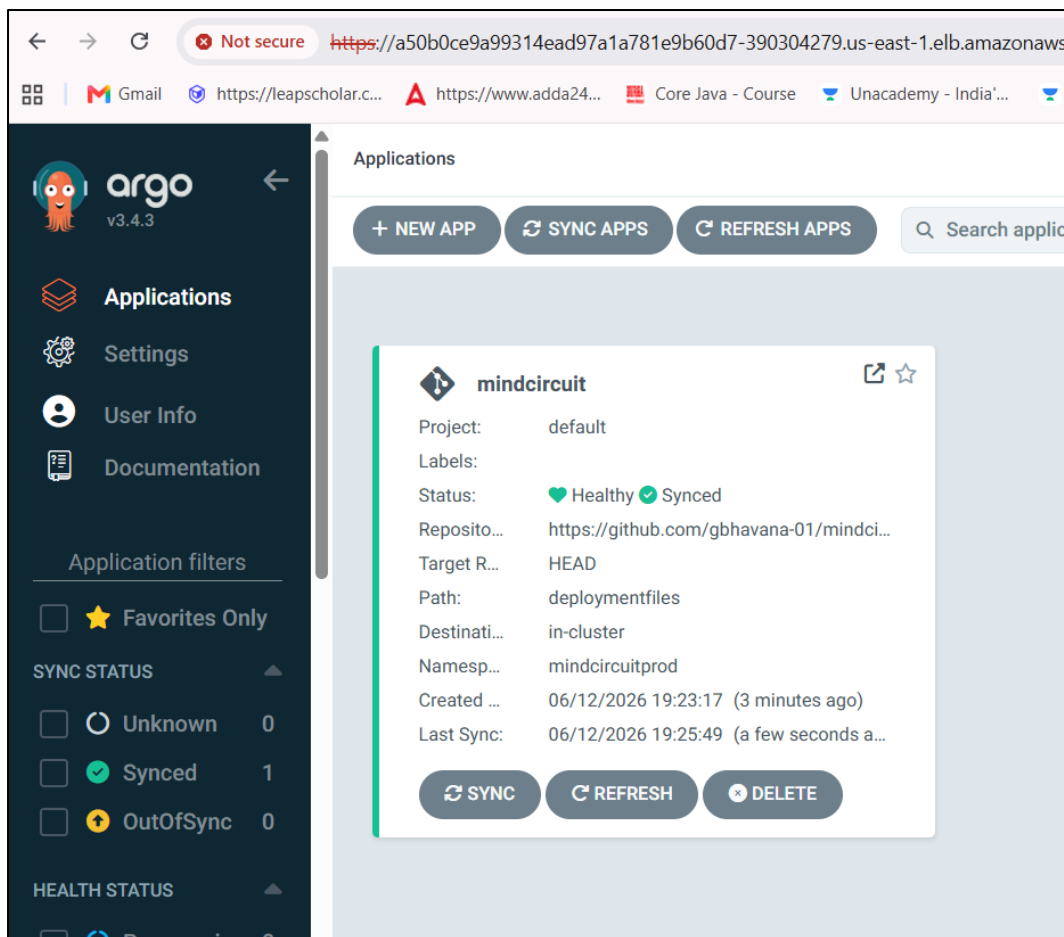


Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

```
$ kubectl create namespace lab
```

namespace/lab created





Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

```
$ kubectl get svc -n lab
```

| NAME       | TYPE         | CLUSTER-IP   | EXTERNAL-IP                                                            | PORT(S)          |
|------------|--------------|--------------|------------------------------------------------------------------------|------------------|
| fb-service | LoadBalancer | 10.100.231.5 | a1ba21f9798f04acc84f6e849bd9cca7-644309754.us-east-1.elb.amazonaws.com | 80:30641/TCP 23s |

fb-service LoadBalancer 10.100.231.5 a1ba21f9798f04acc84f6e849bd9cca7-644309754.us-east-1.elb.amazonaws.com 80:30641/TCP 23s

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops\_MC\_B17D/Kubernetes (main)

\$ kubectl get pods -n lab

| NAME                      | READY | STATUS  | RESTARTS | AGE |
|---------------------------|-------|---------|----------|-----|
| fb-deploy-8df787dcb-44klw | 1/1   | Running | 0        | 64s |
| fb-deploy-8df787dcb-g8kqp | 1/1   | Running | 0        | 64s |
| fb-deploy-8df787dcb-zdt9g | 1/1   | Running | 0        | 64s |

[root@utilserver ~]# kubectl get svc -n lab

| NAME       | TYPE         | CLUSTER-IP   | EXTERNAL-IP                                                            | PORT(S)          |
|------------|--------------|--------------|------------------------------------------------------------------------|------------------|
| fb-service | LoadBalancer | 10.100.231.5 | a1ba21f9798f04acc84f6e849bd9cca7-644309754.us-east-1.elb.amazonaws.com | 80:30641/TCP 73s |

To See the application in lab ns – pods – application – loadbalancer endpoint

a1ba21f9798f04acc84f6e849bd9cca7-644309754.us-east-1.elb.amazonaws.com

**MY NAME IS Madhu Kiran , A PASSIONATE DEVOPS Trainer and LEARNER**

